

There are only two kinds of programming languages: those people always bitch about and those nobody uses.

- Bjarne Stroustrup

CSE341 Programming Languages

Gebze Technical University Computer Engineering Department
2024-2025 Fall Semester
Syntax and Semantics
© 2012-2024 Yakup Genç

Largely adapted from V. Shmatikov, J. Mitchell and R.W. Sebesta

This week....

- Machine Models, Computability and ...
- Grammars

October 2024

CSE341 Lecture – Syntax and Semantics

2

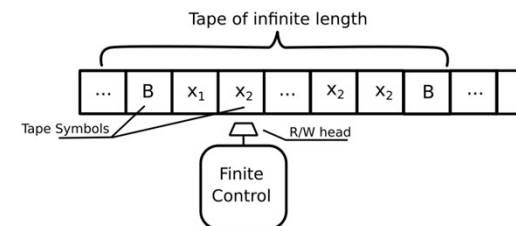
Machine Models and Computing

October 2024

CSE341 Lecture – Syntax and Semantics

3

Turing Machine



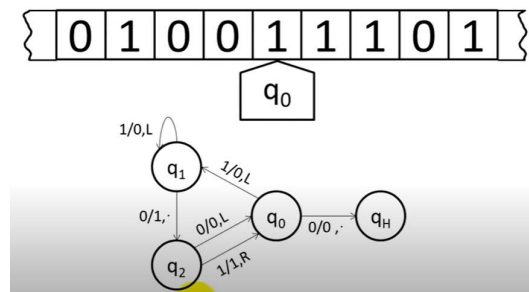
From <https://iq.opengenus.org/general-introduction-to-turing-machine/> without permission.

October 2024

CSE341 Lecture – Syntax and Semantics

4

Turing Machine



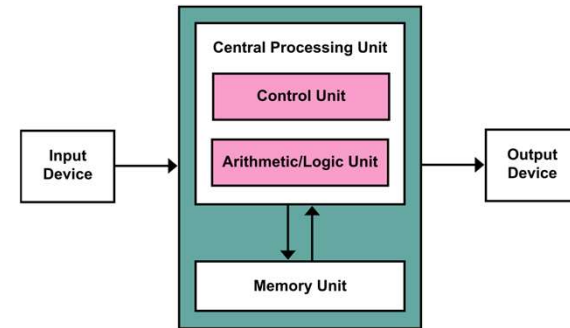
<https://www.youtube.com/watch?v=gJQTFkhkhwPA>

October 2024

CSE341 Lecture – Syntax and Semantics

5

Von Neumann Model

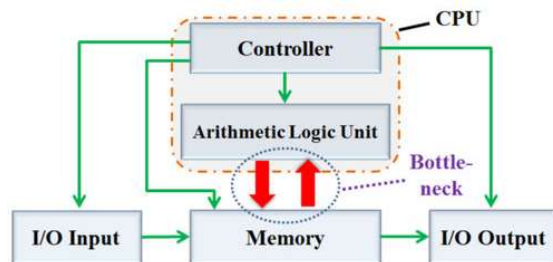


October 2024

CSE341 Lecture – Syntax and Semantics

6

Von Neumann Bottleneck



October 2024

CSE341 Lecture – Syntax and Semantics

7

Von Neumann Programming Languages

- PLs that are high-level abstract isomorphic copies of von Neumann architectures
- The isomorphism between von Neumann programming languages and architectures is in the following manner:
 - program variables $\leftrightarrow$ computer storage cells
 - control statements $\leftrightarrow$ computer test-and-jump instructions
 - assignment statements $\leftrightarrow$ fetching, storing instructions
 - expressions $\leftrightarrow$ memory reference and arithmetic instructions

October 2024

CSE341 Lecture – Syntax and Semantics

8

What is Computation

- Solving a problem $\rightarrow$ function
- A decision problem: well-defined question about well-specified data that has a yes/no answer.
 - For example, primality problem

October 2024

CSE341 Lecture – Syntax and Semantics

9

9

Partial and Total Functions

- Value of an expression may be undefined
 - Undefined operation, e.g., division by zero
 - $3/0$ has no value
 - Implementation may halt with error condition
 - Nontermination
 - $f(x) = \text{if } x=0 \text{ then } 1 \text{ else } f(x-2)$
 - This is a partial function: not defined on all arguments
 - Cannot be detected by inspecting expression (why?)
- These two cases are “mathematically” equivalent, but operationally different (why?)

October 2024

CSE341 Lecture – Syntax and Semantics

10

10

Partial and Total Functions

- Total function $f: A \rightarrow B$ is a subset $f \subseteq A \times B$ with
 - $\forall x \in A$, there is some $y \in B$ with $\langle x, y \rangle \in f$ (total)
 - $\langle x, y \rangle \in f$ and $\langle x, z \rangle \in f$ then $y = z$ (single-valued)
- Partial function $f: A \rightarrow B$ is a subset $f \subseteq A \times B$ with
 - If $\langle x, y \rangle \in f$ and $\langle x, z \rangle \in f$ then $y = z$ (single-valued)
- Programs define partial functions for two reasons
 - What are these reasons?

October 2024

CSE341 Lecture – Syntax and Semantics

11

11

Partial and Total Functions

Programs Define Partial Functions

In most programming languages, it is possible to define functions recursively. For example, here is a function f defined in terms of itself:

```
f(x:int) = if x = 0 then 0 else x + f(x-2);
```

If this were written as a program in some programming language, the declaration would associate the function name f with an algorithm that terminates on every even $x \geq 0$, but diverges (does not halt and return a value) if x is odd or negative. The algorithm for f defines the following mathematical function f , expressed here as a set of ordered pairs:

$$f = \{(x, y) \mid x \text{ is positive and even, } y = 0 + 2 + 4 + \dots + x\}.$$

This is a partial function on the integers. For every integer x , there is at most one y with $f(x) = y$. However, if x is an odd number, then there is no y with $f(x) = y$. Where the algorithm does not terminate, the value of the function is undefined. Because a function call may not terminate, this program defines a partial function.

October 2024

CSE341 Lecture – Syntax and Semantics

12

12

Partial and Total Functions

Computability theory establishes some important ground rules for programming language design and implementation. The following main concepts from this short overview should be remembered:

- *Partiality*: Recursively defined functions may be partial functions. They are not always total functions. A function may be partial because a basic operation is not defined on some argument or because a computation does not terminate.
- *Computability*: Some functions are computable and others are not. Programming languages can be used to define computable functions; we cannot write programs for functions that are not computable in principle.
- *Turing completeness*: All standard general-purpose programming languages give us the same class of computable functions.
- *Undecidability*: Many important properties of programs cannot be determined by any computable function. In particular, the halting problem is undecidable.

October 2024

CSE341 Lecture – Syntax and Semantics

13

13

Partial and Total Functions

When the value of a function or the value of an expression is undefined because a basic operation such as division by zero does not make sense, a compiler or interpreter can cause the program to halt and report the error. However, the undecidability of the halting problem implies that there is no way to detect and report an error whenever a program is not going to halt.

October 2024

CSE341 Lecture – Syntax and Semantics

14

14

Computability

- Function f is computable if some program P computes it
 - For any input x , the computation $P(x)$ halts with output $f(x)$
 - Partial recursive functions: partial functions (int to int) that are computable

October 2024

CSE341 Lecture – Syntax and Semantics

15

15

Halting Function

- Decide whether program halts on input :

Given program P and input x to P ,

$$H(P, x) = \begin{cases} \text{yes,} & \text{if } P(x) \text{ halts} \\ \text{no,} & \text{otherwise} \end{cases}$$

- Clarifications
 - Assume program P requires one string input x
 - Write $P(x)$ for output of P when run with input x
 - Program P is a string input to H
- Fact: There is no program for H

October 2024

CSE341 Lecture – Syntax and Semantics

16

16

Unsolvability of the Halting Problem

- Suppose P solves variant of halting problem
On input Q, assume $P(Q) = \text{yes}$, if $Q(Q)$ halts
no, otherwise
- Build program D
 $D(Q) = \begin{matrix} \text{run forever} & \text{if } Q(Q) \text{ halts} \\ \text{halt} & \text{if } Q(Q) \text{ runs forever} \end{matrix}$
- If $D(D)$ halts, then $D(D)$ runs forever
- If $D(D)$ runs forever, then $D(D)$ halts
- Contradiction! Thus P cannot exist.

17

17

Main Points About Computability

- Some functions are computable, some are not
 - Example: halting problem
- Programming language implementation
 - Can report error if program result is undefined due to an undefined basic operation (e.g., division by zero)
 - Cannot report error if program will not terminate

18

18

Computation Rules

- The factorial function type declaration does not convey how the computation is to proceed
 $\text{fact: int} \rightarrow \text{int}$
- We also need a computation rule
 $\text{fact}(0) = 1$
 $\text{fact}(n) = n * \text{fact}(n-1)$
- This notation is more computationally oriented and can almost be executed by a machine

19

19

Factorial Functions

- C, C++, Java:
 - `int fact (int n) { return (n == 0) ? 1 : n * fact (n-1); }`
- Scheme:
 - `(define fact (lambda (n) (if (= n 0) 1 (* n (fact (- n 1)))))`
- Haskell:
 - `fact :: Integer->Integer`
 - `fact 0 = 1`
 - `fact n = n*fact(n-1)`

20

20

Syntax and Semantics

October 2024

CSE341 Lecture – Syntax and Semantics

21

21

Syntax and Semantics of Programs

- Syntax
 - The symbols used to write a program
- Semantics
 - The actions that occur when a program is executed
- Programming language implementation
 - Syntax → Semantics
 - Transform program syntax into machine instructions that can be executed to cause the correct sequence of actions to occur

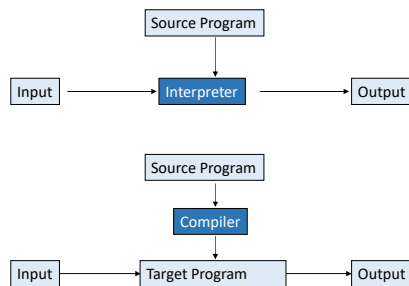
October 2024

CSE341 Lecture – Syntax and Semantics

22

22

Interpreter vs Compiler



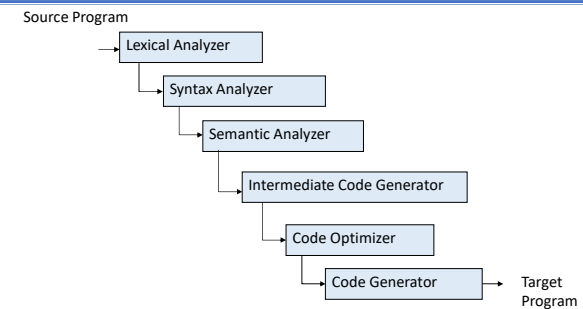
October 2024

CSE341 Lecture – Syntax and Semantics

23

23

Typical Compiler



October 2024

CSE341 Lecture – Syntax and Semantics

24

24

Syntax

- Syntax of a programming language is a precise description of all grammatically correct programs
 - Precise formal syntax was first used in ALGOL 60
- Lexical syntax
 - Basic symbols (names, values, operators, etc.)
- Concrete syntax
 - Rules for writing expressions, statements, programs
- Abstract syntax
 - Internal representation of expressions and statements, capturing their “meaning” (i.e., semantics)

October 2024

CSE341 Lecture – Syntax and Semantics

25

Grammars

- A **meta-language** is a language used to define other languages
- A **grammar** is a meta-language used to define the syntax of a language. It consists of:
 - Finite set of terminal symbols
 - Finite set of non-terminal symbols
 - Finite set of production rules
 - Start symbol
 - Language = (possibly infinite) set of all sequences of symbols that can be derived by applying production rules starting from the start symbol

Backus-Naur
Form (BNF)

John Backus & Peter Naur Panini

October 2024

CSE341 Lecture – Syntax and Semantics

26

Example: Decimal Numbers

- Grammar for unsigned decimal integers
 - Terminal symbols: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
 - Non-terminal symbols: Digit, Integer
 - Production rules:
 - Integer $\rightarrow$ Digit | Integer Digit
 - Digit $\rightarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
 - Start symbol: Integer
- Can derive any unsigned integer using this grammar
 - Language = set of all unsigned decimal integers

October 2024

CSE341 Lecture – Syntax and Semantics

27

Derivation of 352 as an Integer

Integer $\rightarrow$ Integer Digit
 $\rightarrow$ Integer 2
 $\rightarrow$ Integer Digit 2
 $\rightarrow$ Integer 5 2
 $\rightarrow$ Digit 5 2
 $\rightarrow$ 3 5 2

Rightmost derivation

At each step, the rightmost non-terminal is replaced

Production rules:
 Integer $\rightarrow$ Digit | Integer Digit
 Digit $\rightarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

October 2024

CSE341 Lecture – Syntax and Semantics

28

Derivation of 352 as an Integer

Integer $\rightarrow$ Integer Digit
 $\rightarrow$ Integer Digit Digit
 $\rightarrow$ Digit Digit Digit
 $\rightarrow$ 3 Digit Digit
 $\rightarrow$ 3 5 Digit
 $\rightarrow$ 3 5 2

At each step, the leftmost non-terminal is replaced

Leftmost derivation

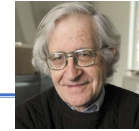
Production rules:
 Integer $\rightarrow$ Digit | Integer Digit
 Digit $\rightarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

October 2024

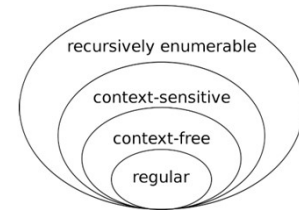
CSE341 Lecture – Syntax and Semantics

29

Chomsky Hierarchy



- Regular grammars
 - Regular expressions, finite-state automata
 - Used to define lexical structure of the language
- Context-free grammars
 - Non-deterministic pushdown automata
 - Used to define concrete syntax of the language
- Context-sensitive grammars
 - Unrestricted grammars
 - Recursively enumerable languages, Turing machines



October 2024

CSE341 Lecture – Syntax and Semantics

30

Regular Grammars

- Generate regular languages
- Left regular grammar
 - All production rules have the form $A \rightarrow \omega$ or $A \rightarrow B\omega$ or $A \rightarrow \epsilon$
 - Here A, B are non-terminal symbols, ω is a terminal symbol
- Right regular grammar
 - $A \rightarrow \omega$ or $A \rightarrow \omega B$ or $A \rightarrow \epsilon$
- Regular expressions (compact notations for regular grammars)
- Example: grammar of decimal integers

October 2024

CSE341 Lecture – Syntax and Semantics

31

Lexical Analysis

- Source code = long string of ASCII characters
- Lexical analyzer splits it into **tokens**
 - Token = sequence of characters (symbolic name) representing a single terminal symbol
- Identifiers: myVariable ...
- Literals: 123 5.67 true ...
- Keywords: char sizeof ...
- Operators: + - * / ...
- Punctuation: ; , { }
- Discards whitespace and comments

October 2024

CSE341 Lecture – Syntax and Semantics

32

Regular Expressions

- `x` character `x`
- `\x` escaped character, e.g., `\n`
- `{ name }` reference to a name
- `M | N` M or N
- `M N` M followed by N
- `M*` 0 or more occurrences of M
- `M+` 1 or more occurrences of M
- `[x1 ... xn]` One of x₁ ... x_n
 - Example: `[aeiou]` – vowels, `[0-9]` - digits

October 2024

CSE341 Lecture – Syntax and Semantics

33

Examples of Tokens in C

- Lexical analyzer usually represents each token by a unique integer code
 - `"+"` { return(PLUS); } // PLUS = 401
 - `"-"` { return(MINUS); } // MINUS = 402
 - `"*"` { return(MULT); } // MULT = 403
 - `"/"` { return(DIV); } // DIV = 404
- Some tokens require regular expressions
 - `[a-zA-Z_][a-zA-Z0-9_]*` { return (ID); } // identifier
 - `[1-9][0-9]*` { return(DECIMALINT); }
 - `0[0-7]*` { return(OCTALINT); }
 - `(0x|0X)[0-9a-fA-F]+` { return(HEXINT); }

October 2024

CSE341 Lecture – Syntax and Semantics

34

Example Lex Program

```
%{
#include<stdio.h>
int flag = 1;
%}

enum {INT, ID, KEYW, UNK};

DIGIT [0-9]
LETTERS [a-zA-Z]

const | break | else | while | for | if
{ printf(" %s is a keyword\n", yytext);
  return(KEYW); }

{LETTERS}+[a-zA-Z0-9]*
{ printf(" %s: identifier\n", yytext);
  return(ID); }

{DIGIT}+
{ printf(" %s: integer\n", yytext);
  return(INT); }

.*
{ printf(" %s: unknown\n", yytext);
  return(UNK); }

int main(){
  printf("Enter something...\n");
  yylex();
}
```

October 2024

CSE341 Lecture – Syntax and Semantics

35

Reserved Keywords in C

- *auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, wchar_t, while*
- C++ added a bunch: *bool, catch, class, dynamic_cast, inline, private, protected, public, static_cast, template, this, virtual* and others
- Each keyword is mapped to its own token

October 2024

CSE341 Lecture – Syntax and Semantics

36

Automatic Scanner Generation

- **Lexer** or **scanner** recognizes and separates lexical tokens
 - Parser usually calls lexer when it's ready to process the next symbol (lexer remembers where it left off)
- Scanner code usually generated automatically
 - Input: lexical definition (e.g., regular expressions)
 - Output: code implementing the scanner
 - Typically, this is a **deterministic finite automaton** (DFA)
 - Examples: Lex, Flex (C and C++), JLex (Java)

October 2024

CSE341 Lecture – Syntax and Semantics

37

Finite State Automata

- Set of states
 - Usually represented as graph nodes
- Input alphabet + unique “end of program” symbol
- State transition function
 - Usually represented as directed graph edges (arcs)
 - Automaton is **deterministic** if, for each state and each input symbol, there is at most one outgoing arc from the state labeled with the input symbol → uniqueness of computation
- Unique start state
- One or more final (accepting) states

October 2024

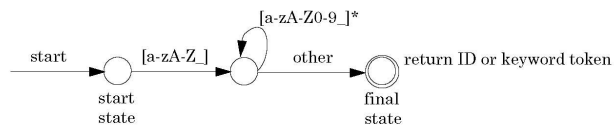
CSE341 Lecture – Syntax and Semantics

38

37

38

DFA for C Identifiers



October 2024

CSE341 Lecture – Syntax and Semantics

39

Traversing a DFA

- **Configuration** = state + remaining input
- **Move** = traversing the arc exiting the state that corresponds to the leftmost input symbol, thereby consuming it
- If no such arc, then...
 - If no input and state is final, then accept
 - Otherwise, error
- Input is **accepted** if, starting with the start state, the automaton consumes all the input and halts in a final state

October 2024

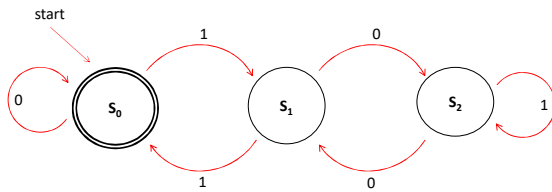
CSE341 Lecture – Syntax and Semantics

40

39

40

Traversing a DFA – Example



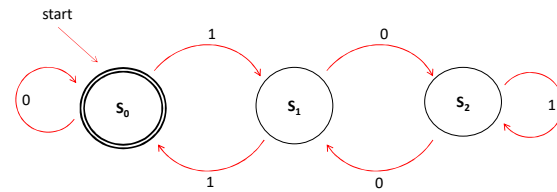
A DFA that accepts only binary numbers that are multiples of 3. The state s_0 is both the start state and an accept state.

October 2024

CSE341 Lecture – Syntax and Semantics

41

Traversing a DFA – Example



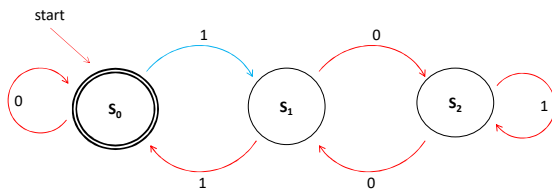
Input: 1001

October 2024

CSE341 Lecture – Syntax and Semantics

42

Traversing a DFA – Example



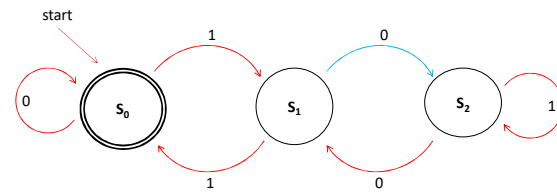
Input: 1001

October 2024

CSE341 Lecture – Syntax and Semantics

43

Traversing a DFA – Example



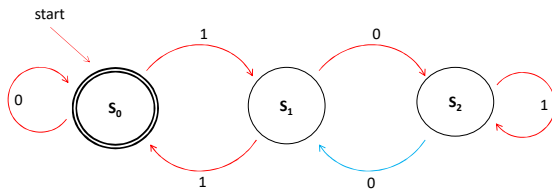
Input: 1001

October 2024

CSE341 Lecture – Syntax and Semantics

44

Traversing a DFA – Example



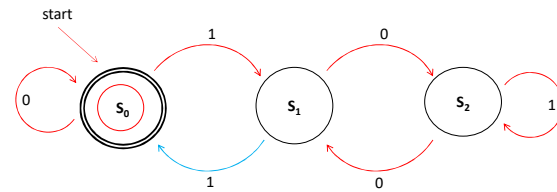
Input: 1001

October 2024

CSE341 Lecture – Syntax and Semantics

45

Traversing a DFA – Example



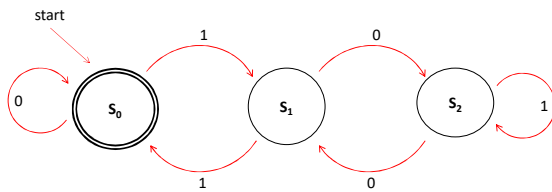
Input: 1001

October 2024

CSE341 Lecture – Syntax and Semantics

46

Traversing a DFA – Example



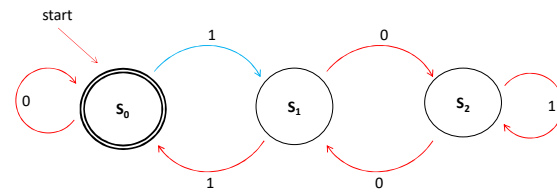
Input: 101

October 2024

CSE341 Lecture – Syntax and Semantics

47

Traversing a DFA – Example



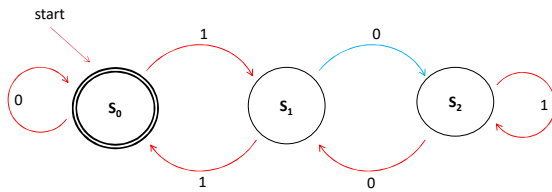
Input: 101

October 2024

CSE341 Lecture – Syntax and Semantics

48

Traversing a DFA – Example



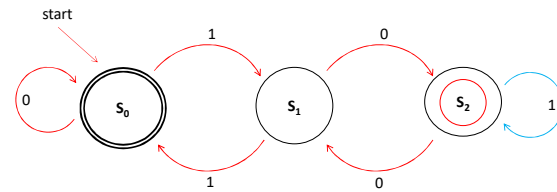
Input: 101

October 2024

CSE341 Lecture – Syntax and Semantics

49

Traversing a DFA – Example

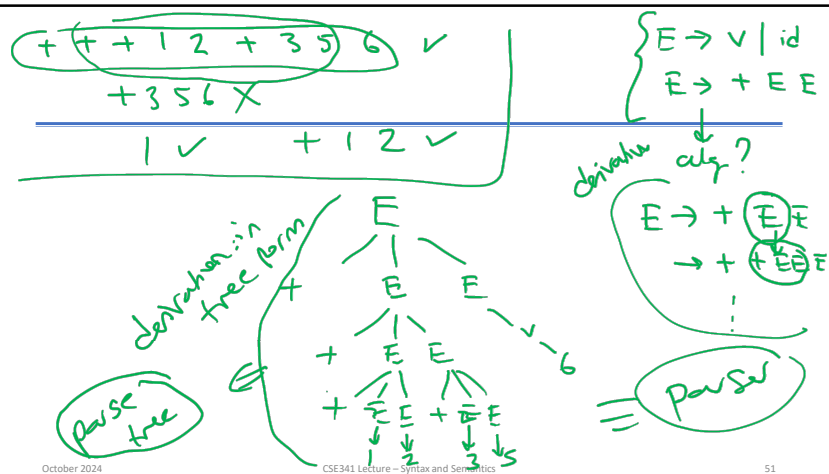


Input: 101

October 2024

CSE341 Lecture – Syntax and Semantics

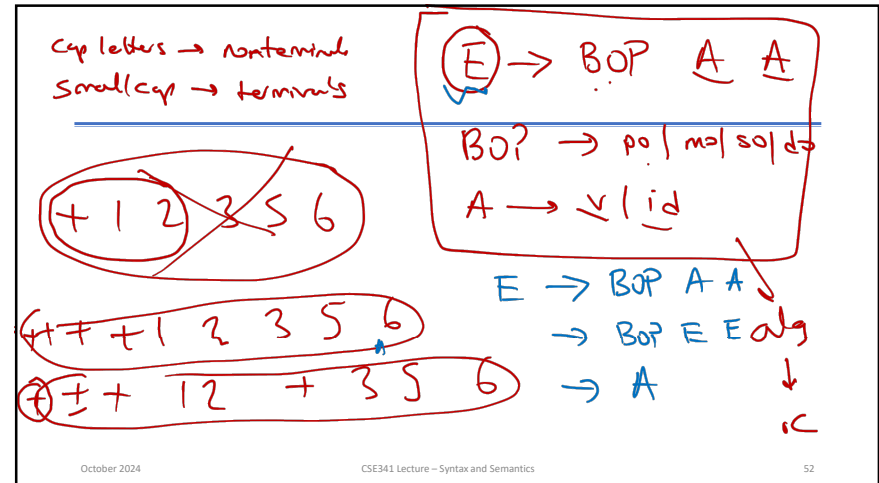
50



October 2024

CSE341 Lecture – Syntax and Semantics

51



October 2024

CSE341 Lecture – Syntax and Semantics

52

49

50

51

52

Context-Free Grammars

- Used to describe concrete syntax
 - Typically using BNF notation
- Production rules have the form $A \rightarrow \omega$
 - A is a non-terminal symbol, ω is a string of terminal and non-terminal symbols
- Parse tree** = graphical representation of derivation
 - Each internal node = LHS of a production rule
 - Internal node must be a non-terminal symbol
 - Children nodes = RHS of this production rule
 - Each leaf node = terminal symbol (token) or "empty"

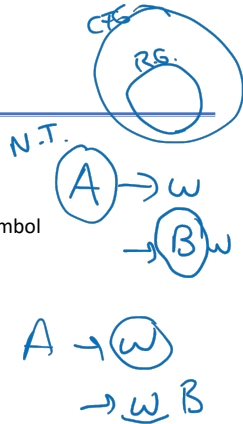
October 2024

CSE341 Lecture – Syntax and Semantics

53

Regular Grammars

- Left regular grammar
 - All production rules have the form $A \rightarrow \omega$ or $A \rightarrow B\omega$
 - Here A, B are non-terminal symbols, ω is a terminal symbol
- Right regular grammar
 - $A \rightarrow \omega$ or $A \rightarrow \omega B$
- Example: grammar of decimal integers



October 2024

CSE341 Lecture – Syntax and Semantics

54

Context-Free Grammars

- CFG is a set of recursive productions rules used to generate patterns of strings
- Consists of:
 - a set of terminal symbols (the characters of the alphabet that appear in the strings generated by the grammar)
 - a set of non-terminal symbols (placeholders for patterns of terminal symbols that can be generated by the non-terminal symbols)
 - a set of productions (rules for replacing non-terminal symbols (on the left side of the production) in a string with other non-terminal or terminal symbols (on the right side of the production))
 - a start symbol (special non-terminal symbol that appears in the initial string generated by the grammar)

October 2024

CSE341 Lecture – Syntax and Semantics

55

Context-Free Grammars

To generate a string of terminal symbols from a CFG:

- Begin with a string consisting of the start symbol;
- Apply one of the productions with the start symbol on the left hand side, replacing the start symbol with the right hand side of the production;
- Repeat the process of selecting non-terminal symbols in the string, and replacing them with the right hand side of some corresponding production, until all non-terminals have been replaced by terminal symbols.

October 2024

CSE341 Lecture – Syntax and Semantics

56

Syntactic Correctness

- Lexical analyzer produces a stream of tokens
- Parser** (syntactic analyzer) verifies that this token stream is syntactically correct by constructing a valid parse tree for the entire program
 - Unique parse tree for each language construct
 - Program = collection of parse trees rooted at the top by a special start symbol
- Parser can be built automatically from the BNF description of the language's CFG
 - Example tools: yacc, Bison



October 2024

CSE341 Lecture – Syntax and Semantics

57

57

CFG For Floating Point Numbers

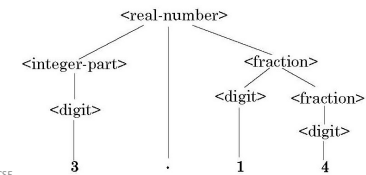
```

<real-number> ::= <integer-part> '.' <fraction-part>
<integer-part> ::= <digit> | <integer-part> <digit>
<fraction> ::= <digit> | <digit> <fraction>
<digit> ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'

```

::= stands for production rule
 <...> are non-terminals
 | represents alternatives for the right-hand side of a production rule

Sample parse tree:



October 2024

CSE

58

58

Grammars and Derivations

Application of a sequence of rules...

```

<program> ::= begin <stmt_list> end
<stmt_list> ::= <stmt> ; <stmt_list>
<stmt> ::= <var> = <expr>
<var> ::= A | B | C
<expr> ::= <var> + <var> | <var> * <var> | <var>

```

Each string in the derivation including <program> is called a **sentential form**

```

<program>
  => begin <stmt_list> end
  => begin <stmt> ; <stmt_list> end
  => begin <var> = <expr> ; <stmt_list> end
  => begin A = <expr> ; <stmt_list> end
  => begin A = <var> + <var> ; <stmt_list> end
  => begin A = B + <var> ; <stmt_list> end
  => begin A = B + C ; <stmt_list> end
  => begin A = B + C ; <stmt> end
  => begin A = B + C ; <var> = <expr> end

```

October 2024

CSE341 Lecture – Syntax and Semantics

59

59

grammar $\xrightarrow{\text{parse}}$ possibly 2+ different Parse Trees $\Rightarrow$ ambiguous..

Grammars define hierarchical syntactic structure $\rightarrow$ parse trees

```

<assign> ::= <id> = <expr>
<id> ::= A | B | C
<expr> ::= <id> + <expr> | <id> * <expr> | ( <expr> ) | <id>

```

A = B * (A + C) can be generated by leftmost derivation...

```

<assign>
  => <id> = <expr>
  => A = <expr>
  => A = <id> * <expr>
  => A = B * <expr>
  => A = B * ( <expr> )
  => A = B * ( <id> + <expr> )
  => A = B * ( A + <expr> )
  => A = B * ( A + <id> )
  => A = B * ( A + C )

```

deriving else?

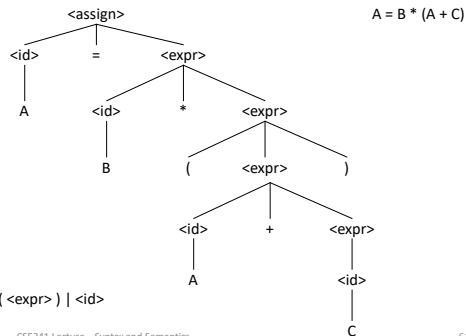
October 2024

CSE341 Lecture – Syntax and Semantics

60

60

Parse Trees



October 2024

CSE341 Lecture – Syntax and Semantics

61

Parsers

- **Parsers:**
 - Determine if the input program is syntactically correct
 - Produce a parse tree for the correct input program
- Classify parsers by the order in which they build the parse tree:
 - Top-down
 - Bottom-up

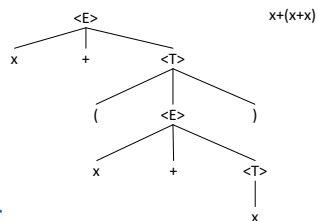
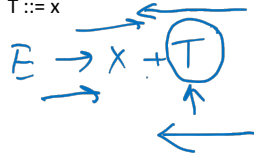
October 2024

CSE341 Lecture – Syntax and Semantics

62

Recursive Descent Parsing

- Perform a depth-first search of the derivation tree for the input being parsed – top-down
- Example:
 - $E ::= x + T$
 - $T ::= (E) \rightarrow$
 - $T ::= x$



October 2024

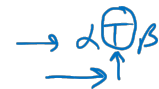
CSE341 Lecture – Syntax and Semantics

63

LL Algorithms

 $t_1 t_2 t_3 \dots t_n$

- Recursive descent – coded directly from BNF
- Parsing table – do not implement BNF rules
- Both are versions of **LL algorithms**
 - Works on same subset of all CFGs
 - 1st L: left to right scan of input
 - 2nd L: leftmost derivation is generated



October 2024

CSE341 Lecture – Syntax and Semantics

64

LL ($\ominus$)
LR ($\perp$)

LR Parsing $\xrightarrow{\text{scanning}}$

Bottom-up parsing

LR algorithms

- Left to right scan of input
- R: rightmost derivation is generated

Python

```
def add(x, y):
    return x+y
```

C

```
int add(int x, int y) {
    return (x+y);
}
```

$t_1, t_2, \dots, t_n$

$\alpha \circ \beta$

$\alpha \circ \beta$

Machine Code Save!!

October 2024 CSE341 Lecture – Syntax and Semantics 65

65

τ id { } - - }

Concrete vs. Abstract Syntax

- Different languages have different concrete syntax for representing expressions, but expressions with common meaning have the same **abstract syntax**

– C: $a+b*c$ Forth: $bc*a+$ (reverse Polish notation)

This expression tree represents the abstract “meaning” of expression

```

    +
   / \
  a   *
     / \
    b   c
  
```

Assumes certain operator precedence (why?)
Not the same as parse tree (why?)
Does the value depend on traversal order?

program:
end:

October 2024 CSE341 Lecture – Syntax and Semantics 66

66

Expression Notation

Inorder traversal $(3+4)*5=35$

```

    *
   / \
  +   5
 / \
3  4
  
```

$3+(4*5)=23$

```

    +
   / \
  3   *
     / \
    4   5
  
```

When constructing expression trees, we want inorder traversal to produce correct arithmetic result based on operator precedence and associativity

Postorder traversal $3\ 4\ 5\ *\ +=35$ $3\ 4\ 5\ *\ +=23$

Easily evaluated using **operand stack** (example: Forth)

Leaf node: push operand value on the stack

Non-leaf binary or unary operator: pop two (resp. one) values from stack, apply operator, push result back on the stack

End of evaluation: print top of the stack

October 2024 CSE341 Lecture – Syntax and Semantics 67

67

Mixed Expression Notation

result = $-b + \text{sqrt}(b*b - 4.0 * a * c);$

unary prefix operators

Prefix:

```

    result
     / \
    +   =
   / \
  -   sqrt
 / \
b  -
   / \
  b  *
     / \
    4.0 a
  
```

result + $-1\ b\ \text{sqrt}\ -2\ *\ b\ b\ *\ * 4.0\ a\ c$

Need to indicate arity to distinguish between unary and binary minus

October 2024 CSE341 Lecture – Syntax and Semantics 68

68

Postfix, Prefix, Mixfix in Java and C

- Increment and decrement: $x++$, $--y$
 $x = ++x + x++$ legal syntax, undefined semantics!
- Ternary conditional
 $(\text{conditional-expr}) ? (\text{then-expr}) : (\text{else-expr});$
 - Example:
 $\text{int min(int a, int b) \{ return (a < b) ? a : b; \}}$
 - This is an expression, NOT an if-then-else command
 - What is the type of this expression?

October 2024

CSE341 Lecture – Syntax and Semantics

69

69

Expression Compilation Example

```
float position, initial, rate;
position = initial + rate * 60;
```

lexical analyzer

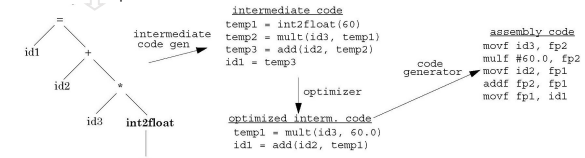
```
[ID, "position"] [ASSIGN, "="] [ID, "initial"] [PLUS, "+"] [ID, "rate"] [MULT, "*"] [NUM, 60] [SEMICOLON, ";"]
```

tokenized expression:

```
id1 = id2 + id3 * 60
```

implicit type conversion (why?)

parser



October 2024

CSE341 Lecture – Syntax and Semantics

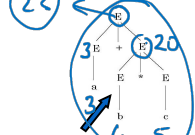
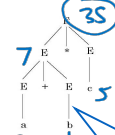
70

70

$E \rightarrow E + E \mid E * E \mid a \mid b \mid c$

① Syntactic Ambiguity

$\langle \text{expr} \rangle ::= \langle \text{expr} \rangle + \langle \text{expr} \rangle \mid \langle \text{expr} \rangle * \langle \text{expr} \rangle \mid a \mid b \mid c$

How to parse $a+b*c$ using this grammar?Parse Tree from a rightmost derivation starting from $\langle \text{expr} \rangle + \langle \text{expr} \rangle$ Parse Tree from a leftmost derivation starting with $\langle \text{expr} \rangle * \langle \text{expr} \rangle$ 

Both parse trees are syntactically valid

Only this tree is semantically correct (operator precedence and associativity are semantic, not syntactic rules)

Problem: this tree is syntactically correct, but semantically incorrect

This grammar is ambiguous

October 2024

CSE341 Lecture – Syntax and Semantics

71

71

Sentential Form

- For a grammar G , with start symbol S , any derivation $S \Rightarrow \alpha$ is called a **sentential form**:
 - If contains only terminal symbols, is a sentence in $L(G)$
 - If contains one or more non-terminals, it is just a sentential form (not a sentence in $L(G)$)
- A **left-sentential form** is a sentential form that occurs in the leftmost derivation of some sentence
- A **right-sentential form** is a sentential form that occurs in the rightmost derivation of some sentence
- E.g.:
 - $\langle P \rangle \Rightarrow \text{begin } \langle \text{stmt_list} \rangle \text{ end}$
 - $\langle P \rangle \Rightarrow \text{begin } A = \langle \text{var} \rangle + \langle \text{var} \rangle; \langle \text{stmt_list} \rangle \text{ end}$
 - $\langle P \rangle \Rightarrow \text{begin } A = B + C; B = C \text{ end}$

October 2024

CSE341 Lecture – Syntax and Semantics

72

72

Derivation Order

- Leftmost derivation: the replaced non-terminal is always the left-most non-terminal in the previous sentential form
 - $\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$
 - $\langle \text{assign} \rangle \Rightarrow A = \langle \text{expr} \rangle$
 - $\langle \text{assign} \rangle \Rightarrow A = \langle \text{id} \rangle * \langle \text{expr} \rangle$
 - $\langle \text{assign} \rangle \Rightarrow A = B * \langle \text{expr} \rangle$
 - $\langle \text{assign} \rangle \Rightarrow A = B * \langle \text{id} \rangle$
 - $\langle \text{assign} \rangle \Rightarrow A = B * C$
- Rightmost derivation: other way around...
 - $\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$
 - $\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{id} \rangle * \langle \text{expr} \rangle$
 - $\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{id} \rangle * \langle \text{id} \rangle$
 - $\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{id} \rangle * C$
 - $\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = B * C$
 - $\langle \text{assign} \rangle \Rightarrow A = B * C$
- Derivation order has no effect on the language generated by grammar

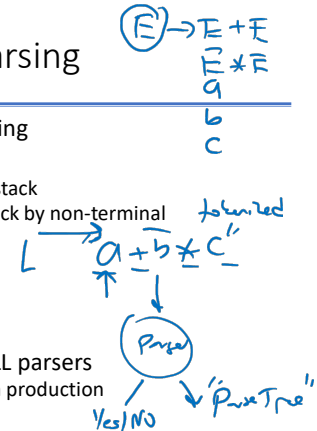
October 2024

CSE341 Lecture – Syntax and Semantics

73

Shift-Reduce Parsing

- Stack implementation of shift-reduce parsing
- Four possible action
 - Shift** – move the next input symbol to top of stack
 - Reduce** – replace the handle on the top of stack by non-terminal
 - Accept** – successful completion of parsing
 - Error** – syntax error discovered
- Initial stack empty...
- End of input is empty...
- Note: LR parsers are more powerful than LL parsers
 - they can see the entire RHS before choosing a production



October 2024

CSE341 Lecture – Syntax and Semantics

74

Shift-Reduce Parsing

- The parser repeatedly matches the right-hand side (RHS) of a production against a substring in the current right-sentential form
- At each match, it applies a reduction to build the parse tree:
 - each reduction replaces the matched substring with the nonterminal on the left-hand side (LHS) of the production
 - each reduction adds an internal node to the current parse tree
 - the result is another right-sentential form
- The final result is a rightmost derivation, in reverse

October 2024

CSE341 Lecture – Syntax and Semantics

75

Handles

- We are trying to find a substring α of the current right-sentential form where:
 - matches some production $A := \alpha$
 - reducing α to A is one step in the reverse of a rightmost derivation
- Call such a string a handle

October 2024

CSE341 Lecture – Syntax and Semantics

76

Handle

- A handle of a right-sentential form γ is a pair $\langle A \Rightarrow \beta, k \rangle$ where $A \Rightarrow \beta$ is a production rule and k is the position in γ of β 's rightmost symbol.
- If $\langle A \Rightarrow \beta, k \rangle$ is a handle, then replacing β in γ at position k with A produces the previous right-sentential form from which γ is derived in a rightmost derivation
 - $S \Rightarrow \alpha A \omega \Rightarrow \alpha \beta \omega$
- Because γ is a right-sentential form, the substring to the right of handle contains only terminal symbols $\Rightarrow$ the parser does not need to scan past the handle (only lookahead)
- If the grammar is **unambiguous**, then every right sentential form of the grammar has exactly one handle

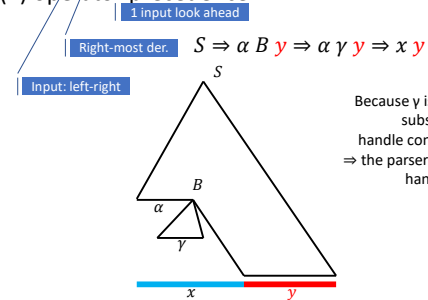
October 2024

CSE341 Lecture – Syntax and Semantics

77

LR(1) Parsing

- LR(1) operator precedence



October 2024

CSE341 Lecture – Syntax and Semantics

78

Shift-Reduce Parsing

$E ::= E + T \mid T$
 $T ::= T * F \mid F$
 $F ::= (E) \mid id$

Step	Parse Stack	Right-most Sentential	LookAhead	Unscanned	Parser Action
0	empty		id	id+id*id	Shift
1	id	id+id*id	+	+id*id	Reduce
2	F	F+id*id	+	+id*id	Reduce
3	T	T+id*id	+	+id*id	Reduce
4	E	E+id*id	+	+id*id	Shift
5	E+	E+id*id	id	id*id	Shift
6	E+id	E+id*id	*	*id	Reduce
7	E+F	E+F*id	*	*id	Reduce
8	E+T	E+T*id	*	*id	Shift
9	E+T*	E+T*id	id	id	Shift
10	E+T*id	E+T*id	empty	empty	Reduce
11	E+T*F	E+T*F	empty	empty	Reduce
12	E+T	E+T	empty	empty	Reduce
13	E	E	empty	empty	Accept

October 2024

CSE341 Lecture – Syntax and Semantics

79

Shift-Reduce Parsing

$E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow a$
 $E \rightarrow b$
 $E \rightarrow c$

Step	Parse Stack	Right-most Sentential	LookAhead	Unscanned	Parser Action
0			'a'	a+b*c	Shift
1	@		+	+b*c	Reduce
2	E		+	+b*c	
3	E+				
4	E+*				
5	E+E				
6	E				
7	E*				
8	E*b				
9	E*c				
10	E				
11	E				
12					
13					

1. $a+b*c$

October 2024

CSE341 Lecture – Syntax and Semantics

80

Expression $\rightarrow$ generate a value!!

$:= \sim \rightarrow$

Shift-Reduce Parsing

$E ::= i \mid v \mid E E \mid E +$

$E \rightarrow i$

$3 \ 2 \ +$

$1 \times + y *$

Step	Parse Stack	Right-most Derivation	Look Ahead	Unscanned	Parser Action
0					
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					

October 2024 CSE341 Lecture – Syntax and Semantics 81

81

Parse Tree for this Example

$E ::= E + T \mid T, T ::= T * F \mid F, F ::= (E) \mid id$

$id + id * id$

```

graph TD
    E --> E1[E]
    E --> P1[+]
    E --> T1[T]
    E1 --> T2[T]
    T2 --> F1[F]
    F1 --> id1[id]
    T1 --> P2[*]
    T1 --> F2[F]
    T1 --> F3[F]
    F2 --> T3[T]
    T3 --> F4[F]
    F4 --> id2[id]
    F3 --> id3[id]
  
```

October 2024 CSE341 Lecture – Syntax and Semantics 82

82

Conflicts

- Generic shift-reduce strategy:
 - If there is a handle on top of the stack, reduce
 - Otherwise, shift
- But what if there is a choice?
 - If it is legal to shift or reduce, there is a
 - shift-reduce conflict
 - If it is legal to reduce by two different productions, there is a reduce-reduce conflict
- Source of conflicts:
 - Ambiguous grammars always cause conflicts
 - So do many non-ambiguous grammars

$reg_{exp} \rightarrow .l \rightarrow (lex$

$(b) \rightarrow .y$

$\rightarrow -c$

$\rightarrow \circ$

$\rightarrow .c$

October 2024 CSE341 Lecture – Syntax and Semantics 83

83

Removing Ambiguity

- Define a distinct non-terminal symbol for each operator precedence level
- Define RHS of production rule to enforce proper associativity
- Extra non-terminal for smallest subexpressions

$\langle expr \rangle ::= \langle expr \rangle + \langle expr \rangle \mid \langle expr \rangle * \langle expr \rangle \mid a \mid b \mid c$

$E ::= E + T \mid E - T \mid T$

$T ::= T * F \mid T / F \mid F$

$F ::= (E) \mid id \mid num$

Not always possible to remove ambiguity this way!

October 2024 CSE341 Lecture – Syntax and Semantics 84

84

This Grammar Is Unambiguous

```

E ::= E + T | E - T | T
T ::= T * F | T / F | F
F ::= ( E ) | id | num

```

Leftmost:

```

E => E + T
=> T + T
=> E + T
=> id + T
=> id + T * F
=> id + E * F
=> id + id * F
=> id + id * id

```

Rightmost:

```

E => E + T
=> E + T * F
=> E + T * id
=> E + E * id
=> E + id * id
=> T + id * id
=> E + id * id
=> id + id * id

```

October 2024

CSE341 Lecture – Syntax and Semantics

85

85

Left- and Right-Recursive Grammars

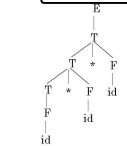
```

E ::= E + T | E - T | T
T ::= T * F | T / F | F
F ::= ( E ) | id | num

```

Leftmost non-terminal on the RHS of production is the same as the LHS

Left recursive parse tree for $id * id * id$ with left-associativity



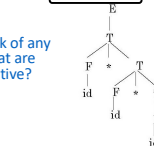
```

E ::= T + E | T - E | T
T ::= F * T | F / T | F
F ::= ( E ) | id | num

```

Right-recursive grammar

Right recursive parse tree for $id * id * id$ with right-associativity



Can you think of any operators that are right-associative?

October 2024

CSE341 Lecture – Syntax and Semantics

86

86

Operator Associativity

- Needed in the absence of parentheses for operators on both sides of an operand, e.g., $\dots^a a^a \dots$
- Right associative: operators are grouped from right
 - $5^4 4^3 2 \Rightarrow 5^{(4^{(3^2)})}$
- Left-associative: operators are grouped from left
 - $5 + 4 + 3 + 2 \Rightarrow ((5 + 4) + 3) + 2$
- **left-associative**: addition, subtraction, multiplication, and division
- **right-associative**: exponentiation, assignment and conditional

October 2024

CSE341 Lecture – Syntax and Semantics

87

87

Right-Associative Exponentiation

```

<factor> ::= <exp> ** <factor> | <exp>
<exp> ::= (<expr>)| <id>

```

October 2024

CSE341 Lecture – Syntax and Semantics

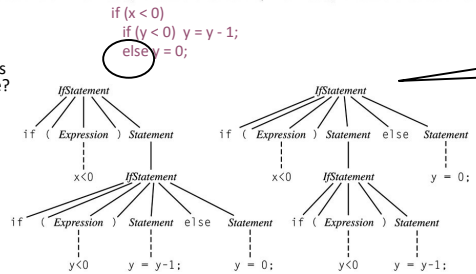
88

88

“Dangling Else” Ambiguity

`stmt ::= if expr then stmt | if expr then stmt else stmt`

With which `if` does this `else` associate?



Classic example of a shift-reduce conflict

October 2024

CSE341 Lecture – Syntax and Semantics

89

89

Solving the Dangling Else Ambiguity

- Algol 60, C, C++: associate each `else` with closest `if`; use `{ ... }` or `begin ... end` to override
- Algol 68, Modula, Ada: use an explicit delimiter to end every conditional (e.g., `if ... endif`)
- Java: rewrite the grammar and restrict what can appear inside a nested `if` statement
 - `IfThenStmt` $\rightarrow$ `if ( Expr ) Stmt`
 - `IfThenElseStmt` $\rightarrow$ `if ( Expr ) StmtNoShortIf else Stmt`
 - The category `StmtNoShortIf` includes all except `IfThenStmt`

October 2024

CSE341 Lecture – Syntax and Semantics

90

90

Removing Ambiguity – Factoring

- Ambiguous grammar:

`S ::= if B then S`

`S ::= if B then S else S`

- Factoring:

`S ::= if B then S Z`

`Z ::= ;`

`Z ::= else S`

October 2024

CSE341 Lecture – Syntax and Semantics

91

91

Removing Ambiguity – Factoring

- Given

`A ::=  $\alpha \beta_1$  |  $\alpha \beta_2$  | ... |  $\alpha \beta_n$`

- Invent a new non-terminal `F` and replace

`A ::=  $\alpha F$`

`F ::=  $\beta_1$  |  $\beta_2$  | ... |  $\beta_n$`

October 2024

CSE341 Lecture – Syntax and Semantics

92

92

Removing Ambiguity – Substitution

- Given all B productions on LHS ...

...

$A ::= B \alpha$

$B ::= \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$

...

- Replace $A ::= B \alpha$ with

$A ::= \beta_1 \alpha \mid \beta_2 \alpha \mid \dots \mid \beta_n \alpha$

October 2024

CSE341 Lecture – Syntax and Semantics

93

93

EBNF

- Extending BNF to improve on minor inconveniences
- Use brackets for optional part on RHS
 - $\langle \text{if_stmt} \rangle ::= \text{if } \langle \text{expr} \rangle \langle \text{stmt} \rangle [\text{else } \langle \text{stmt} \rangle]$
- Use braces on RHS for indefinite repeat or none
 - $\langle \text{ident_list} \rangle ::= \langle \text{identifier} \rangle \{ , \langle \text{identifier} \rangle \}$
- Multiple choice option
 - $\langle \text{term} \rangle ::= \langle \text{term} \rangle (* \mid / \mid \%) \langle \text{factor} \rangle$
 - $\langle \text{term} \rangle ::= \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{term} \rangle / \langle \text{factor} \rangle \mid \langle \text{term} \rangle \% \langle \text{factor} \rangle$
- Meta-symbols: $\{, [], ()$

October 2024

CSE341 Lecture – Syntax and Semantics

94

94

More Powerful Grammars

- Context-sensitive: production rules have the form
 - $\alpha A \beta \rightarrow \alpha \omega \beta$
 - A is a non-terminal symbol, α, β, ω are strings of terminal and non-terminal symbols
 - Deciding whether a string belongs to a language generated by a context-sensitive grammar is PSPACE-complete
 - Emptiness of a language is [undecidable](#)
- Unrestricted: equivalent to Turing machine

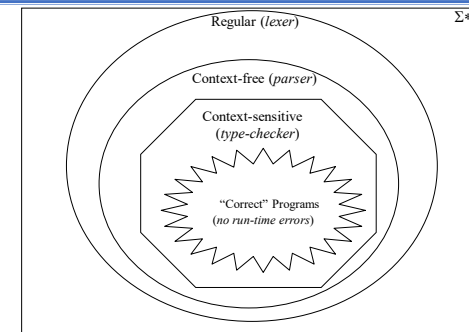
October 2024

CSE341 Lecture – Syntax and Semantics

95

95

Grammars



October 2024

CSE341 Lecture – Syntax and Semantics

96

96

Semantic Analysis

- Beyond context free grammar
 - Is x declared before it is used?
 - Is x declared but never used?
 - Is an expression type consistent?
 - Is an array reference in bounds?
 - ...
- Choices
 - Use context sensitive grammars
 - Hard to define and costly to use
 - Use attribute grammar
 - Can help to some extent
 - Use ad hoc methods
 - Mostly required

October 2024

CSE341 Lecture – Syntax and Semantics

97

97

CFG $\rightarrow$ BNF $\leftarrow$ Attribute
 complete Attribute Grammars abstract

- Augment context free grammar with rules
 - Each grammar symbol has an associated set of attributes
 - The attributes are evaluated by the semantic rules attached to the productions
- Syntax directed translation
 - Use attribute grammar for semantic analysis
 - Type checking
 - Generate intermediate code or representation
- Static Semantics
 - Not for running of a program, but for legal forms of programs

October 2024

CSE341 Lecture – Syntax and Semantics

98

98

Attribute Grammar

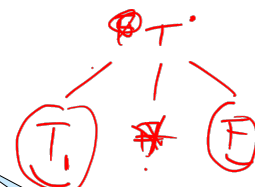
Example

Expression evaluation

$L \rightarrow E$ $\text{print}(E.\text{val})$
 $E \rightarrow E_1 + T$ $E.\text{val} := E_1.\text{val} + T.\text{val}$
 $E \rightarrow T$ $E.\text{val} := T.\text{val}$
 $T \rightarrow T_1 * F$ $T.\text{val} := T_1.\text{val} * F.\text{val}$
 $T \rightarrow F$ $T.\text{val} := F.\text{val}$
 $F \rightarrow (E)$ $F.\text{val} := E.\text{val}$
 $F \rightarrow \text{digit}$ $F.\text{val} := \text{digit.lexval}$

Predicate functions: static semantic rule
(production rule in BNF)

Attribute computation
~ semantic functions...



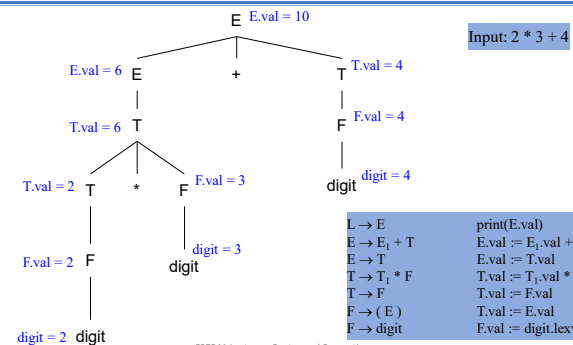
October 2024

CSE341 Lecture – Syntax and Semantics

99

99

Attribute Grammar and Parse Tree



October 2024

CSE341 Lecture – Syntax and Semantics

100

100

Semantics and Attributed Grammar

Semantics

- Give the meaning of the language
- BNF

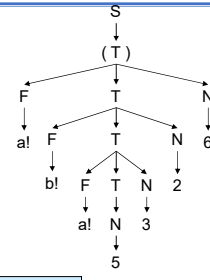
$S \rightarrow (T), T \rightarrow F T N \mid N$
 $F \rightarrow a! \mid b!, N \rightarrow \text{num}$
 • Input: (a! b! a! 5 3 2 6)

With the attributed grammar

$S \rightarrow (T)$ $\text{print}(T.\text{list})$
 $T \rightarrow N$ $T.\text{list} := [N.\text{val}]$
 $T \rightarrow F T_1 N$ if (F.op = a) T.list :=
 append(T₁.list, N.val)
 if (F.op = b) T.list :=
 removeLastN(T₁.list, N.val)

$F \rightarrow a!$ F.op := a
 $F \rightarrow b!$ F.op := b
 $N \rightarrow \text{num}$ N.val := num

Remove last N.val
elements from T₁.list



October 2024

CSE341 Lecture – Syntax and Semantics

101

101

Semantics and Attributed Grammar

Semantics

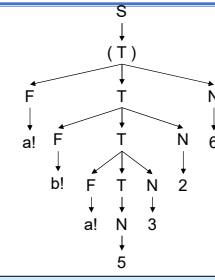
- Give the meaning of the language
- BNF

$S \rightarrow (T), T \rightarrow F T N \mid N$
 $F \rightarrow a! \mid b!, N \rightarrow \text{num}$
 • Input: (a! b! a! 5 3 2 6)

With the attributed grammar

$S \rightarrow (T)$ $\text{print}(T.\text{val})$
 $T \rightarrow F T_1 N$ $T.\text{val} := T_1.\text{val} \text{ E.op } N.\text{val}$
 $T \rightarrow N$ $T.\text{val} := N.\text{val}$
 $F \rightarrow a!$ F.op := +
 $F \rightarrow b!$ F.op := *
 $N \rightarrow \text{num}$ N.val := num

Attributed grammar defines the meaning for
the production rules which gives the meaning
for the input program



October 2024

CSE341 Lecture – Syntax and Semantics

102

102

Build Parse Tree

Parse tree construction

- Can be done with only synthesized attributes:

- child: first child pointer
- sib: right sibling pointer
- addr: parse tree node address

$S \rightarrow AB$ $S.\text{addr} = \text{new-node}(S)$
 $S.\text{child} = A.\text{addr}; A.\text{sib} = B.\text{addr};$
 $A \rightarrow CdE$ $A.\text{addr} = \text{new-node}(A); d.\text{addr} = \text{new-node}(d);$
 $A.\text{child} = C.\text{addr}; C.\text{sib} = d.\text{addr}; d.\text{sib} = E.\text{addr};$
 $A \rightarrow F$ $A.\text{addr} = \text{new-node}(A)$
 $A.\text{child} = F.\text{addr};$
 $B \rightarrow EF$ $B.\text{addr} = \text{new-node}(B)$
 $B.\text{child} = E.\text{addr}; E.\text{sib} = F.\text{addr}$

October 2024

CSE341 Lecture – Syntax and Semantics

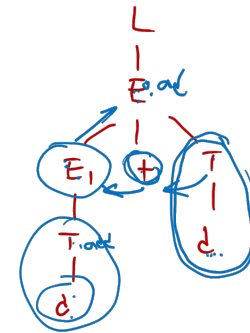
103

103

Build Parse Tree

Parse tree construction example

$L \rightarrow E$ $L.\text{addr} = \text{new-node}(L);$
 $L.\text{child} = E.\text{addr};$
 $E \rightarrow E_1 + T$ $E.\text{addr} = \text{new-node}(E);$
 $E.\text{addr} = \text{new-node}(+);$
 $E.\text{child} = E_1.\text{addr};$
 $E_1.\text{sib} = T.\text{addr};$
 $E \rightarrow T$ $E.\text{addr} = \text{new-node}(E);$
 $E.\text{child} = T.\text{addr};$
 $T \rightarrow \text{digit}$ $T.\text{addr} = \text{new-node}(T);$
 $T.\text{child} = \text{new-node}(\text{digit});$



October 2024

CSE341 Lecture – Syntax and Semantics

104

104

Concrete syntax.

- ① describe
- ② parse

abstract syntax

d_1 - what the ...
 d_2 - division by 0.

$1+3+x**z$
 $\downarrow$
 $v \cup v \cup v \cup v \cup \emptyset$ **AN!**
 $1+3*5 \rightarrow \text{yes!}$

~~$1+3*5$~~ $> 1+3*5 \rightarrow 16.0000$
 $>$

105

Concrete syntax $\rightarrow$ BNF

$E \rightarrow E_1 + T$
 $E \rightarrow T$
 $T \rightarrow d$

Attributes

$t_1 = 1.val$
 $T_1.val = t_1$
 $E_1.val = T_1.val$
 $T_2.val = 3.val$
 $E_2.val = T_2.val$
 $E.val = E_1.val + E_2.val$

$\cdot val \rightarrow$ (abstract syntax)

$1+3$

T_1
 E_1
 T_2
 E_2
 d

power $\rightarrow$ bottom up power!!

$T_1.val = t_1$
 $E_1.val = T_1.val$
 $T_2.val = 3.val$
 $E_2.val = T_2.val$
 $E.val = E_1.val + E_2.val$

$print(E.val);$

106

$E \rightarrow E_1 \oplus T$ E / T $E.type$

$E.val = power(E_1.val, T.val)$

~~$2^0 = 1$~~ **exception** $power = 0$

if ($T.val == 0$) **exit** ($POWERZERO$)
 else $E.val = power(E_1.val, T.val);$

107

shift-reduce $\rightarrow$ bunch of code

$\downarrow$ executed

$print(E.val)$

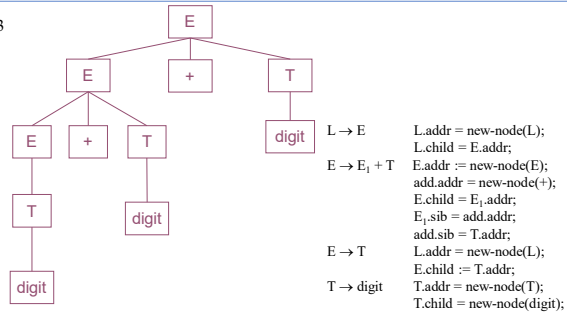
or parse tree showing up $\rightarrow$ E

$1+3.1*5$

108

Build Parse Tree

Input: 1 + 2 + 3



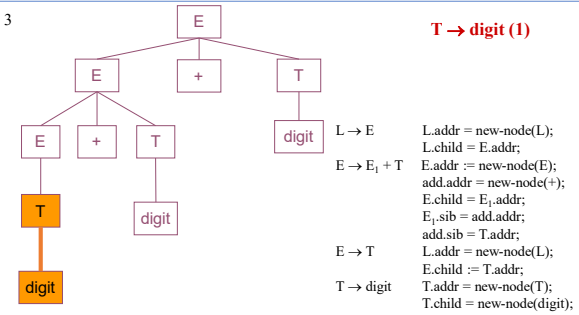
October 2024

CSE341 Lecture – Syntax and Semantics

109

Build Parse Tree

Input: 1 + 2 + 3



October 2024

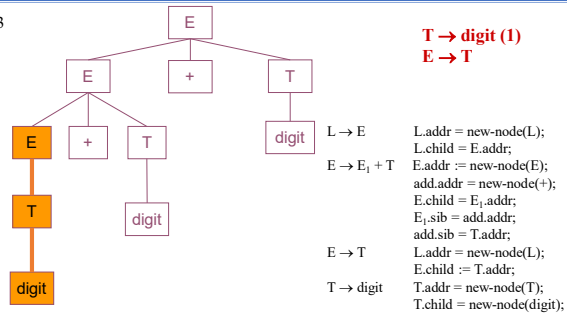
CSE341 Lecture – Syntax and Semantics

110

110

Build Parse Tree

Input: 1 + 2 + 3



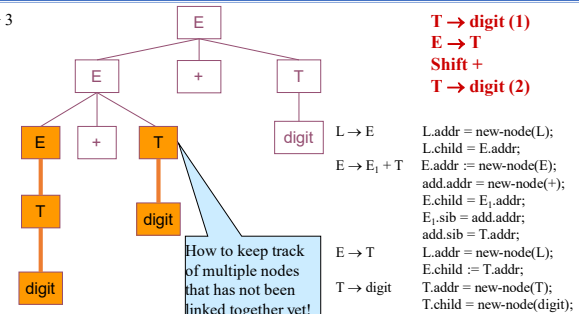
October 2024

CSE341 Lecture – Syntax and Semantics

111

Build Parse Tree

Input: 1 + 2 + 3



October 2024

CSE341 Lecture – Syntax and Semantics

112

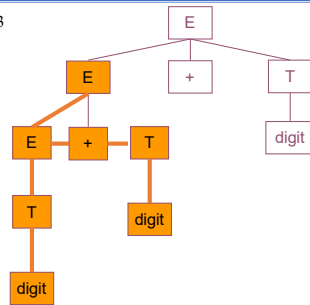
112

109

111

Build Parse Tree

Input: 1 + 2 + 3



$T \rightarrow \text{digit (1)}$
 $E \rightarrow T$
 Shift +
 $T \rightarrow \text{digit (2)}$
 $E \rightarrow E_1 + T$

$L \rightarrow E$
 $L.\text{addr} = \text{new-node}(L);$
 $L.\text{child} = E.\text{addr};$
 $E \rightarrow E_1 + T$
 $E.\text{addr} := \text{new-node}(E);$
 $\text{add.addr} = \text{new-node}(+);$
 $E.\text{child} = E_1.\text{addr};$
 $E_1.\text{sib} = \text{add.addr};$
 $\text{add.sib} = T.\text{addr};$
 $E \rightarrow T$
 $L.\text{addr} = \text{new-node}(L);$
 $E.\text{child} := T.\text{addr};$
 $T \rightarrow \text{digit}$
 $T.\text{addr} = \text{new-node}(T);$
 $T.\text{child} = \text{new-node}(\text{digit});$

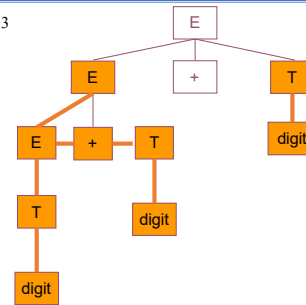
October 2024

CSE341 Lecture – Syntax and Semantics

113

Build Parse Tree

Input: 1 + 2 + 3



$\dots$
 $T \rightarrow \text{digit (2)}$
 $E \rightarrow E_1 + T$
 Shift +
 $T \rightarrow \text{digit (3)}$

$L \rightarrow E$
 $L.\text{addr} = \text{new-node}(L);$
 $L.\text{child} = E.\text{addr};$
 $E \rightarrow E_1 + T$
 $E.\text{addr} := \text{new-node}(E);$
 $\text{add.addr} = \text{new-node}(+);$
 $E.\text{child} = E_1.\text{addr};$
 $E_1.\text{sib} = \text{add.addr};$
 $\text{add.sib} = T.\text{addr};$
 $E \rightarrow T$
 $L.\text{addr} = \text{new-node}(L);$
 $E.\text{child} := T.\text{addr};$
 $T \rightarrow \text{digit}$
 $T.\text{addr} = \text{new-node}(T);$
 $T.\text{child} = \text{new-node}(\text{digit});$

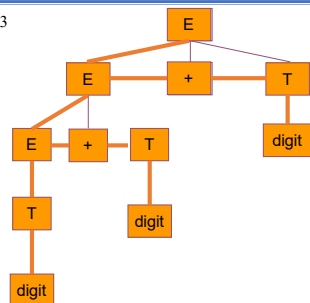
October 2024

CSE341 Lecture – Syntax and Semantics

114

Build Parse Tree

Input: 1 + 2 + 3



$\dots$
 $E \rightarrow E_1 + T$
 Shift +
 $T \rightarrow \text{digit (3)}$
 $E \rightarrow E_1 + T$

$L \rightarrow E$
 $L.\text{addr} = \text{new-node}(L);$
 $L.\text{child} = E.\text{addr};$
 $E \rightarrow E_1 + T$
 $E.\text{addr} := \text{new-node}(E);$
 $\text{add.addr} = \text{new-node}(+);$
 $E.\text{child} = E_1.\text{addr};$
 $E_1.\text{sib} = \text{add.addr};$
 $\text{add.sib} = T.\text{addr};$
 $E \rightarrow T$
 $L.\text{addr} = \text{new-node}(L);$
 $E.\text{child} := T.\text{addr};$
 $T \rightarrow \text{digit}$
 $T.\text{addr} = \text{new-node}(T);$
 $T.\text{child} = \text{new-node}(\text{digit});$

October 2024

CSE341 Lecture – Syntax and Semantics

115

Attribute Grammar

Two types of attributes

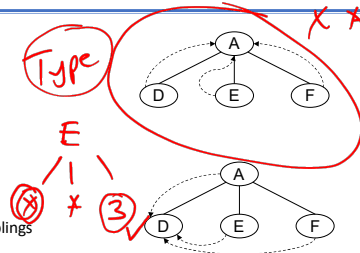
- Synthesized attributes**
 - Attribute values are evaluated from bottom up
 - The value of the parent is defined on the values of the children

Inherited attributes

- Attribute values are evaluated from top down
- The value of a node is defined on the values of its parent and/or siblings

Attribute rules

- Only reference local information
- only refer to symbols in the corresponding production



October 2024

CSE341 Lecture – Syntax and Semantics

116

113

114

115

116

S-Attribute Grammar

- Only has synthesized attributes
- Suitable for shift-reduce parsing
- Example

– Expression evaluation

```

L → E      print(E.val)
E → E1 + T  E.val := E1.val + T.val
E → T      E.val := T.val
T → T1 * F  T.val := T1.val * F.val
T → F      T.val := F.val
F → ( E )  F.val := E.val
F → digit  F.val := digit.lexval

```

Derive parent's value from children's values

Derive parent's value from children's values

lexval: value from lex



October 2024

CSE341 Lecture – Syntax and Semantics

117

Inherited Attributes

• Example

– Adding type to symbol table

```

D → TL      L.type := T.type
T → int     T.type := integer
T → real    T.type := real
L → L1, id  L1.type := L.type; addtype(id.entry, L.type)
L → id      addtype(id.entry, L.type)

```

- id.entry is not defined here
- It is the entry to the symbol table

L's attribute value "type" depends on left sibling T's "type"

Attribute "type" is still synthesized

L's attribute value "type" depends on its parent "type" value ⇒ "type" is an inherited attribute

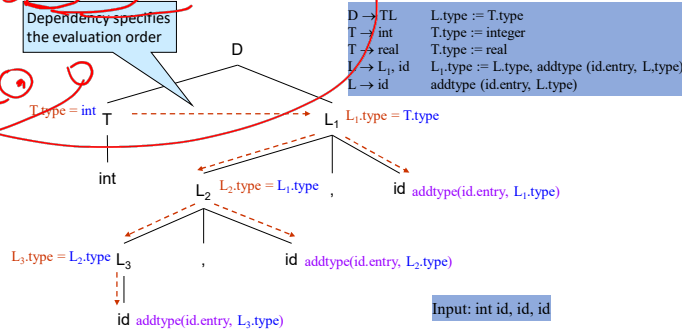
October 2024

CSE341 Lecture – Syntax and Semantics

118

Dependency Graph

Dependency specifies the evaluation order



October 2024

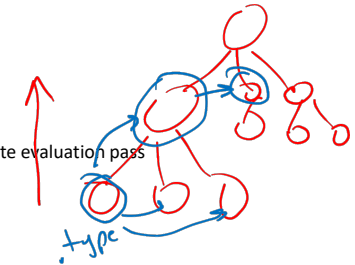
CSE341 Lecture – Syntax and Semantics

119

$$(x) = 1 + y$$

Dependency Graph

- Evaluation based on the dependencies
- Circularity check
 - Dependency graph should be acyclic
- Build pass tree
 - First build the parse tree, then evaluate
 - Cannot be done with parsing, require a separate evaluation pass
- Can we do better?
 - L-attributed grammar
 - Parser stack based technique
 - Marker nonterminals (not covered)
 - Rewrite grammar rules



October 2024

CSE341 Lecture – Syntax and Semantics

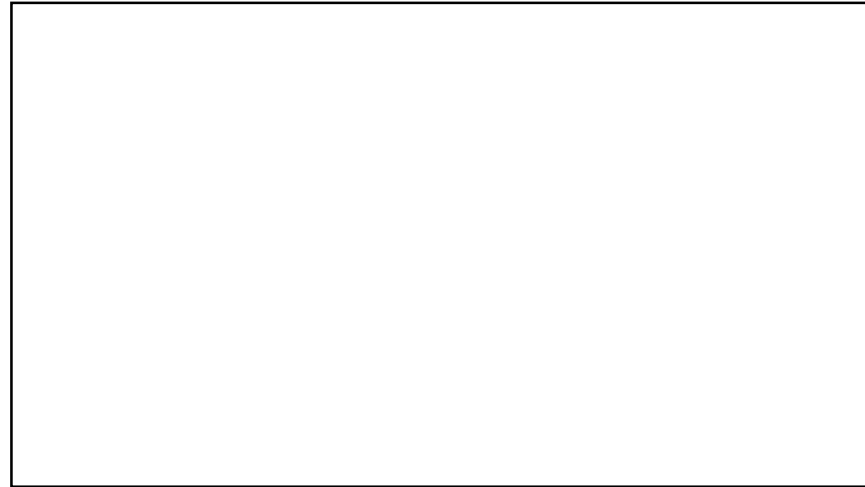
120

117

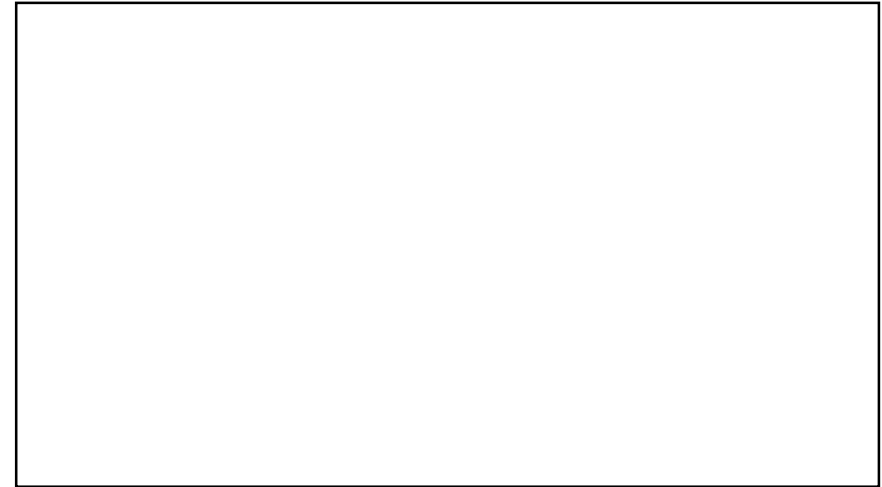
118

119

120



125



126

*(char **) x ++*

Dynamic Semantics

```

#include <stdio.h>
void main() {
    printf("Hello");
}

```

- Describing meaning of expressions, statements, and program units of a PL
- Operational semantics:** describing meaning of a statement or program → specifying the effects of running it on a machine...
 - Natural OS: highest level – what's the final result
 - Structural OS: lowest level – precise meaning of a program by examining the complete sequence of state changes during programs' run
- Do we need another intermediate language to define operational semantics...

October 2024 CSE341 Lecture – Syntax and Semantics 127

127

Operational Semantics

C statement	Meaning
<i>for (expr1; expr2; expr3) {</i>	expr1:
...	loop: if expr2 == 0 go to out
}	...
	expr3:
	goto loop
	out: ...

October 2024 CSE341 Lecture – Syntax and Semantics 128

128

Dynamic Semantics

- Describing meaning of expressions, statements, and program units of a PL
- Denotational semantics**: formal method to describing meaning of programs
 - Based on recursive function theory
 - Mathematical model maps instances of PL's entities to mathematical objects → denotational
 - Domain: syntactic domain *parser output*
 - Range: semantic domain

October 2024

CSE341 Lecture – Syntax and Semantics

129

Denotational Semantics

- Consider the grammar
 - $\langle \text{binnum} \rangle := '0' \mid '1' \mid \langle \text{binnum} \rangle '0' \mid \langle \text{binnum} \rangle '1'$
- Semantic function:
 - $M_{\text{bin}} : \text{BNF} \rightarrow \text{BinaryNumbers}$
- E.g. *in terms of decimal rep?*
 - $M_{\text{bin}}('0') = 0$
 - $M_{\text{bin}}('1') = 1$
 - $M_{\text{bin}}(\langle \text{binnum} \rangle '0') = 2 * M_{\text{bin}}(\langle \text{binnum} \rangle)$
 - $M_{\text{bin}}(\langle \text{binnum} \rangle '1') = 2 * M_{\text{bin}}(\langle \text{binnum} \rangle) + 1$

October 2024

CSE341 Lecture – Syntax and Semantics

130

Thank you for listening!

131

Interpreted – (R E P L)

① Assignment statement

② Expression

$S \rightarrow id = E$

$P \rightarrow S \mid E$
print the value of expr / As. statement

– value/literal → integer
 – variable/identifier → alpha-
 char
 – operators → =, *, +

$E \rightarrow E_1 + T$
 $E \rightarrow T$
 $T \rightarrow T_1 * F$
 $F \rightarrow F_1 / F_2$
 $F \rightarrow lit \mid id$

132

SSS-ex-2

$abc\ 123\ xyz\ =\ =\ =\ * \ +$ $\neq S$
 $\neq E$

lexer: $\{ id, lit, id, op, op, op, op \}$
 $\{ 'abc', '123', 'xyz', '=', '=', '=', '*', '+' \}$

$? \ x\ y\ 12\ 3$
 $\rightarrow$ not in lexeme!! $\rightarrow$ error

$> a = 3$ CS✓

$> a + 3$ CS✓

133

$a = 3$

$P \rightarrow S$
 $P.val = S.val$

$S \rightarrow id = lit$

$\{ id.val = lit.val;$
 $S.val = id.val;$

$\Rightarrow$

$id.val = lit.val$
 $S.val = id.val$
 $P.val = S.val$
 $print("05d", P.val);$

Diagram showing a tree structure with root P and children C_1 , S_1 , S_2 . A red circle highlights the subtree rooted at P with children S_1 and S_2 . Inside the circle, $P.val$ is shown, and S_1 and S_2 are shown with $id.val$ and $lit.val$ respectively.

134

$a + 3$

~~$P \rightarrow E$~~
 $P \rightarrow E$

$F_1.val = id.val;$
 $T_1.val = F_1.val;$
 $E_1.val = T_1.val;$
 $F_2.val = lit.val;$
 $T_2.val = F_2.val;$

Diagram showing a tree structure with root P and children E , $+$, T . E has children E_1 , $+$, T . E_1 has children F_1 , $+$, F . F_1 has children F , $+$, F . F has children id , lit . T has children F , $+$, F . F has children id , lit .

$E.val = E_1.val + T_2.val$
 $P.val = E.val$
 $print("05d", P.val);$

135